CO2 Assessment Tool 1
Scenario Based Assignment

Name:-Arshiya Noor Mohamed
Reg;-192471010
Course Code:- CSA1709 - Artificial Intelligence

1. AI-Powered Drones for Emergency Medicine Delivery

i. Greedy Best-First Search: GBFS selects the location with the lowest heuristic value, such as straight-line distance to the destination. It is fast and explores promising routes, but ignores the actual travel cost. Under uncertainty, it may choose flooded or dangerous paths and cannot guarantee an optimal route.

ii. A Search: A* evaluates routes using **f(n) = g(n) + h(n)**, where g(n) is the actual cost incurred and h(n) is the estimated remaining cost. It balances actual and estimated costs, helping the drone choose efficient routes. With suitable heuristics and stable costs, it can find an optimal path.

iii. Heuristic Accuracy: Accurate heuristics guide both algorithms towards the destination. GBFS depends heavily on heuristic quality, whereas A* also considers accumulated cost. An admissible heuristic that does not overestimate the remaining cost helps A* preserve optimality.

iv. Dynamic Changes: Blocked roads and changing weather require the drone to update its map and replan. Repeated searches increase computation time. Incremental algorithms such as D* Lite reuse previous search information to improve replanning efficiency.

v. Recommendation: A* is suitable for cost-effective routing, while D* Lite is preferable when obstacles and travel costs change frequently. Safety constraints, battery limits and weather risks must also be considered.

---

2. Smart City Traffic Signal Optimization

Problem Formulation: The objective is to minimize total waiting time, fuel consumption and congestion using a weighted cost function: $\mathbf{J = w_1W + w_2F + w_3C}$, where W, F and C represent waiting time, fuel consumption and congestion. Constraints include safe signal durations, pedestrian crossings and traffic regulations. Traditional exhaustive search is inefficient because the number of possible timing combinations grows rapidly.

i. Hill Climbing: Hill Climbing repeatedly modifies signal timings and accepts changes that improve the objective. It is simple, fast and memory-efficient but may become trapped in a local optimum.

ii. Local Optima, Plateaus and Ridges: A local optimum is a solution better than nearby alternatives but not globally best. A plateau is a region where many solutions have similar costs. A ridge occurs when coordinated changes across several intersections are needed for improvement.

iii. Simulated Annealing and Genetic Algorithms: Simulated Annealing sometimes accepts worse solutions to escape local optima, gradually reducing this probability. Genetic Algorithms evolve a population of solutions using selection, crossover and mutation, exploring different combinations of signal timings.

iv. Recommendation: A hybrid Genetic Algorithm with local search is suitable for large-scale optimization. Fast local adjustments should handle real-time traffic changes, while hard safety constraints must always be enforced.

---

3. Autonomous Mars Rover: Online Search

i. Need for Online Search: The rover explores unknown terrain without a complete map. Online search allows it to observe, act, learn and replan during exploration, unlike offline search, which plans using an available model beforehand.

ii. Partial Observability and Dynamic Environments: Limited sensors may not reveal hidden craters, rocks or unstable terrain. Uncertain position estimates, unexpected obstacles and communication delays complicate decisions. The rover must balance safety, energy consumption and sample collection.

iii. Internal Model: The rover collects sensor data, estimates its position, identifies obstacles and builds a local map. It selects an action, moves, compares observations with expectations and updates its map and route.

iv. Search Comparison: Uninformed search, such as Breadth-First Search, uses no heuristic and may explore many unnecessary states. Informed search, such as A*, uses heuristic estimates to guide planning. Online search learns about the environment while acting and can incorporate either search strategy.

v. Recommendation: Online heuristic search with continuous mapping and safety-aware replanning is most suitable. Local A* can plan routes

using known terrain, while updated sensor information triggers replanning. Risk, slope, obstacles and energy consumption should influence route selection.

---

4. University Examination Timetabling as a CSP

Formulation: A Constraint Satisfaction Problem (CSP) consists of variables, domains and constraints. Each examination must be assigned a valid time slot and examination hall.

i. Variables, Domains and Constraints:

- **Variables:** Examinations such as Data Structures and DBMS.
- **Domains:** Available time slots and suitable examination halls.
- **Constraints:** Students must not have overlapping exams; halls must have sufficient capacity and cannot host simultaneous exams; prerequisite exams must follow the required order; faculty availability and special accommodations must be satisfied.

Hard constraints must always be met, while soft constraints, such as minimizing gaps between examinations, should be optimized.

ii. Backtracking Search: The system assigns a time and hall to each examination, checking constraints after every assignment. If a conflict occurs, it reverses the previous assignment and tries another option. This continues until a valid timetable is found or all possibilities are exhausted.

iii. Constraint Propagation: Forward checking removes incompatible time slots from unassigned examinations after each assignment. If a variable has no valid options remaining, the algorithm backtracks immediately. Minimum Remaining Values prioritizes the most

constrained examination, while the degree heuristic prioritizes examinations affecting many others.

iv. Scalability: Large universities face numerous courses, students, halls and changing availability. A constraint-programming solver combining backtracking, forward checking and heuristics is recommended. For very large timetables, decomposition and optimization techniques can improve efficiency, while local re-optimization handles last-minute changes.

---

5. Strategic Game AI: Minimax and Alpha-Beta Pruning

i. Minimax Algorithm: Minimax models competitive games as decision trees. MAX represents the AI, which chooses the highest-value move, while MIN represents the opponent, who chooses the lowest-value outcome for the AI. Values are evaluated at terminal states or depth-limited leaf nodes and propagated upwards to select the best move.

ii. Computational Challenges: If the branching factor is b and search depth is d, Minimax has time complexity $\mathbf{O(b^d)}$. Large game trees require significant computation and memory, making deep searches difficult under strict real-time deadlines.

iii. Alpha-Beta Pruning: Alpha-Beta pruning eliminates branches that cannot affect the final Minimax decision. Alpha (α) is the best value MAX can guarantee, while beta (β) is the best value MIN can guarantee. When $\boldsymbol{\alpha \geq \beta}$, further exploration of that branch is unnecessary. Pruning produces the same result as Minimax at the same search depth. With ideal move ordering, complexity can improve to approximately $\mathbf{O(b^{d/2})}$; in the worst case, it remains $O(b^d)$.

iv. Evaluation Function: A possible function is $\mathbf{Eval(s) = w_1M + w_2R + w_3T + w_4O - w_5D}$, where M represents military strength, R resources, T territory, O objective progress and D damage or vulnerability. Weights reflect the importance of each factor. This function estimates the strategic value of states that cannot be explored to the end of the game.

v. Recommendation: Iterative deepening Minimax with Alpha-Beta pruning and move ordering is suitable for real-time play. Iterative deepening searches progressively greater depths, retaining the best move from the last completed search when time expires. Transposition tables avoid repeated evaluations. This balances computational efficiency and decision quality, although limited-depth search cannot guarantee a globally optimal move.
